UNIVERSITY OF VICTORIA

Department of Electrical and Computer Engineering

ECE 455 – Real Time Computer Systems Design

1. Introduction

This project involved the design and implementation of a real-time Traffic Light System using the STM32F4 Discovery board and FreeRTOS. The objective was to apply real-time OS concepts to model and control vehicle traffic at a simplified one-lane intersection. The system dynamically adjusts traffic flow using a potentiometer input and manages vehicle movement and traffic light timing according to specifications found in the lab manual.

The project was implemented using multiple FreeRTOS tasks, queues for inter-task communication, and a software timer to manage traffic light state transitions. Hardware peripherals including GPIO and the ADC were configured to interface with the LEDs and potentiometer.

This report outlines the design solution, implementation, and limitations and possible improvements relative to the specified functional and technical requirements.

2. Design Solution

Our design of the traffic light solution was focused on utilizing FreeRTOS tasks, queues and timers to make a multi-threaded program sharing resources through queues.

2.1 Initial Software Design

The initial design followed the recommended framework provided in the lab manual, centered around a multi-tasking architecture comprising four distinct FreeRTOS tasks, four Message Queues, and a Software Timer. Within the `main()` function, the system would perform the hardware abstraction layer initialization for the GPIO and ADC peripherals. Following hardware setup, the kernel would initialize the communication queues and instantiate the tasks and timers before invoking the RTOS Scheduler via `vTaskStartScheduler`.

The conceptual flow of the initial solution was structured as follows:

2.1.1 Potentiometer Task

This task was intended to handle the analog input by performing an ADC conversion to sample the potentiometer's voltage. This raw data would then be normalized into a meaningful integer value to represent traffic density and signal duration. To simulate a polling environment, the task would utilize `vTaskDelay` with a 100 ms block time. Given the criticality of user input for system timing, this task was to be assigned the Highest Priority, and the processed value would be inserted into the `xPotQueue`.

2.1.2 Update Display Task

The Update Display task would serve as the primary output interface. It was designed to read the `xTrafficColorQueue` and the `xCarArrayQueue`. The latter would hold a 24-bit data structure where each bit functioned as a boolean flag representing a car (1) or an empty road segment (0). The task would then drive the specific GPIO pins to reflect the current state of both the traffic lights and the vehicle positions on the LED hardware.

2.1.3 Traffic Generator Task

This task was intended to manage the arrival logic for new vehicles. By retrieving the density value from the `xPotQueue`, the task would determine the rate of vehicle entry, where high potentiometer values would correlate to high traffic volume and vice versa. Based on this calculation, the task would determine how many cars were to be queued to enter the simulation and pass this count into the `xCarGeneratedQueue`.

2.1.4 Traffic Light Task

The Traffic Light task would cycle the intersection lights through Green, Yellow, and Red states. The timing for each state transition would be dynamic, adjusting its internal delay based on the flow rate data received from the `xPotQueue`. Upon a state change, the task would update the `xTrafficColorQueue` to notify the Display Task.

2.1.5 Timer

A single FreeRTOS Software Timer was designated to control the global car velocity. Since the requirements dictated a movement speed of approximately one car per second, the timer period would be set to 1000 ms. The timer's Callback Function would contain the primary simulation logic; it would access the `xCarArrayQueue` to perform a bit-shift operation, incorporating logic to stop cars at the intersection during a red light state. After processing the movement, the callback would update the `xCarArrayQueue` so the Display Task could reflect the simulated car progression.

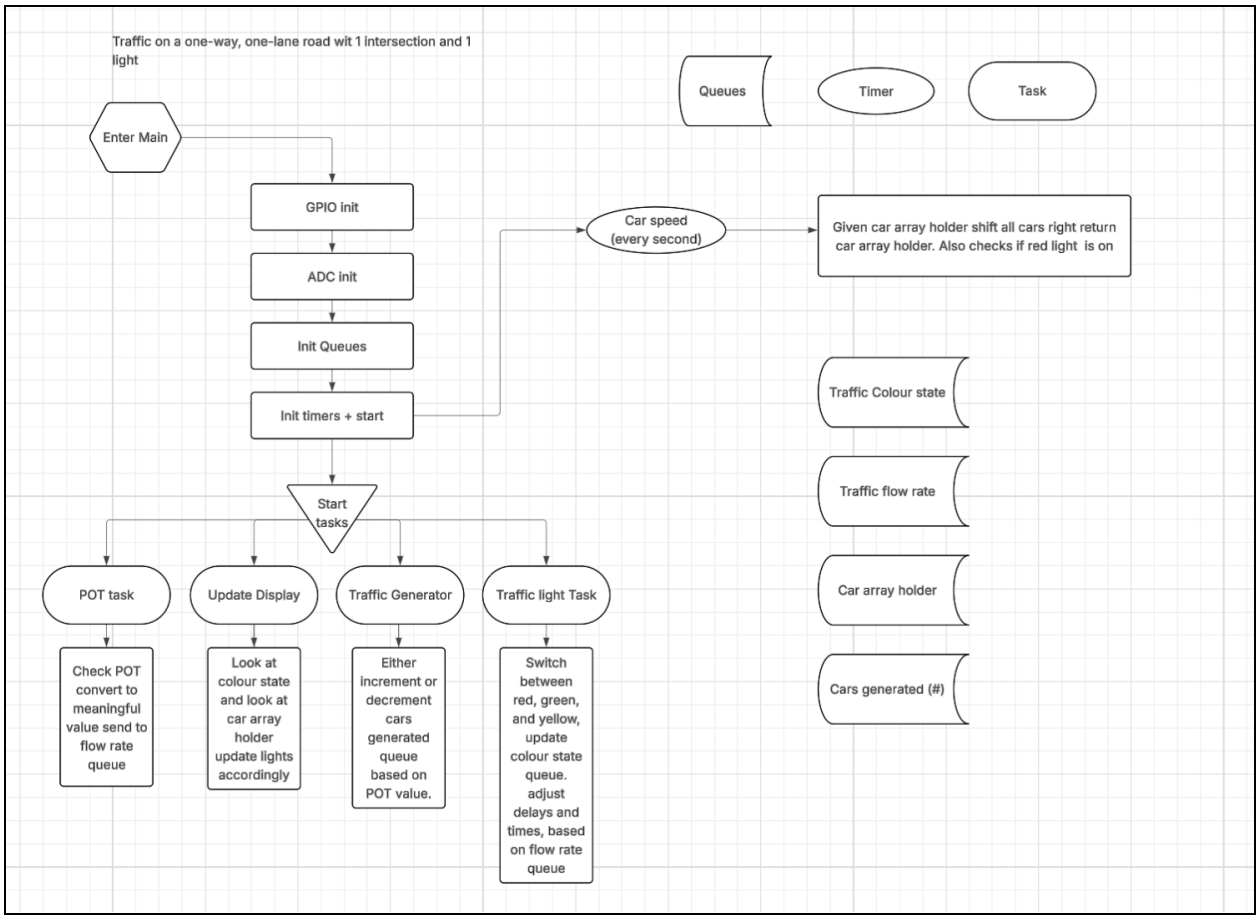

Figure 1. Flowchart of initial design solution.

During the physical implementation phase, several architectural adjustments were required. These changes were necessary to ensure smooth transitions for vehicle movement and to prevent specific tasks from entering a Blocked State. These refinements, along with the final stable implementation, are detailed in Section 2.2.

2.2 Final Software Design

The final solution transitioned from the preliminary four-task model to an optimized architecture comprising three FreeRTOS tasks, three Message Queues, and one Software Timer.

2.2.1 Initialization

The system would begin by executing `prvSetupHardware`, which configured the RCC clocks, GPIO pins for the shift register and traffic lights, and the ADC. Following hardware setup, three queues were set up: `xFlowQueue` to store the ADC value, `xLightQueue` to communicate the current LightState, and `xSpawnQueue` to pass a binary signal indicating if a new vehicle should enter the simulation. During initial testing, all tasks were set to the same priority with

`vTrafficFlowTask` using 100 ms delay, which caused logic blocking. In the final version, the `vTrafficFlowTask` was assigned a higher priority and the delay was increased to 1000 ms to prevent blocking.

2.2.2 Traffic Flow Task

This high-priority task managed the system's primary input. It would trigger an ADC conversion to sample the potentiometer and retrieve the value via `ADC_GetConversionValue()`. To ensure the rest of the system always had access to the most recent sensor data without filling the queue, it utilized `xQueueOverwrite`. The task would then enter a Blocked state for 1000 ms using `vTaskDelay`.

2.2.3 Traffic Density Task

This task implemented the stochastic vehicle generation algorithm. It would use `xQueuePeek` to examine the flow rate from the `xFlowQueue` without removing the data from the queue. The logic involved generating a pseudo-random number between 0 and 99. At maximum traffic density, a car would be generated every second. At minimum density, the threshold was set to 18 percent, resulting in an average arrival rate of one car approximately every 5.5 seconds. The result, either a 1 for spawn or 0 for empty, was then sent to the `xSpawnQueue`.

2.2.4 Display Task

The Display Task managed the 24-bit representation of the roadway and served as the primary output driver. It initialized a local car_array to track vehicle positions. Every 1000 ms, the task would peek at the `xLightQueue` and receive the signal from the `xSpawnQueue`. The movement logic involved calling three helper functions in a specific order: `shift_after_intersection()`, `shift_in_intersection()`, and `shift_before_intersection()`. This sequence ensured that cars moved forward only if the space ahead was clear or if the traffic light was green. The task then called `move_cars()`, which manually toggled the CLOCK, DATA, and RESET GPIO pins to show on the breadboard the cars moving.

2.2.5 Traffic Light Timer

As per the technical requirements, a single FreeRTOS Software Timer managed the traffic signal logic. The timer's behavior was dynamic, with the yellow phase fixed at a constant duration of 2000 ms. The green and red phases used the `calculate_traffic_light_duration()`helper function to scale the period between 2 and 4 seconds based on the potentiometer value. The timer callback utilized `xTimerChangePeriod` to update the duration on the fly and broadcasted the new LightState to the rest of the system via the `xLightQueue`. This approach allowed the tasks to

operate harmoniously, with the potentiometer effectively modulating both the frequency of vehicle arrivals and the timing of the signal phases.

2.3 Hardware Design

The hardware implementation of the Traffic Light System consists of three primary subsystems: the traffic light LEDS, the vehicle display driven by shift registers, and the potentiometer interface for traffic flow control. The system was constructed on a breadboard and interfaced with the STM32F4 Discovery Board.

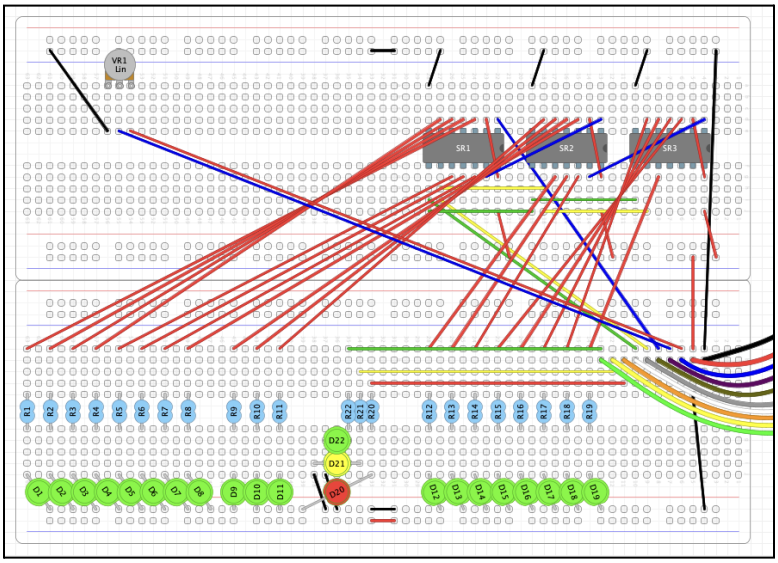

Figure 2. Breadboard layout diagram of the Traffic Light System wiring.

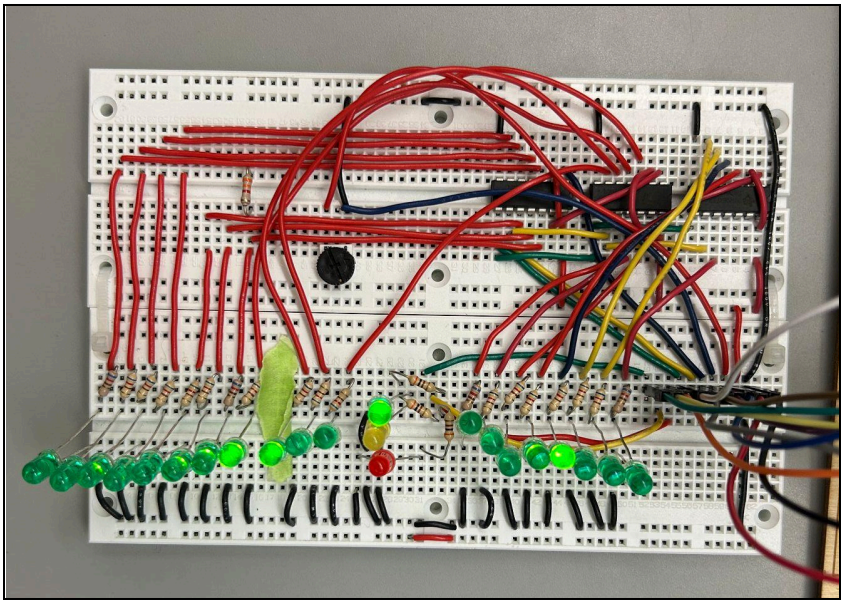

Figure 3. Physical breadboard implementation.

Figure 2 shows the equivalent breadboard layout diagram of the circuit. Figure 3 shows the physical breadboard implementation used during testing and implementation. While the physical wiring includes overlap and slightly different jumper pathing, it is electrically equivalent to the layout diagram shown in Figure 2.

2.3.1 Traffic Light Subsystem

The traffic light is implemented using three discrete LEDs representing red, yellow, and green states. These LEDs are connected to GPIOC pins PC0, PC1, and PC2 respectively. Each LED is connected in series with a resistor.

The GPIO pins are configured as push-pull digital outputs. All LEDs share a common ground. The software timer callback function updates these GPIO outputs to transition between light states.

2.3.2 Vehicle Display and Shift Registers

Vehicle positions are represented using 21 LEDs arranged in a row to simulate traffic flow. To reduce GPIO usage, three serial-in/parrallel-out shift registers are daisy-chained to provide 24 available outputs.

The shift register control signals are connected as follows:

- PC6 - Serial Data
- PC7 - Clock
- PC8 - Reset

The serial output of each shift register feeds the serial input of the next. During each display update cycle, 24 bits are shifted sequentially into the registers. The clock signal advances each bit through the chain, and the reset signal initializes the shifting process.

Each vehicle LED includes a resistor. The use of shift-registers significantly reduces required microcontroller I/O while maintaining full control over vehicle visualization.

2.3.3 Potentiometer and ADC Interface

Traffic density and traffic light timing are controlled using a linear potentiometer connected to PC3. The potentiometer is wired as a voltage divider:

- One terminal connected to 3.3V
- One terminal connected to GND
- The center wiper connected to PC3

The ADC is configured for 12-bit resolution in single conversion mode. This produces a digital value in the range 0-4095, which is used in the software to adjust vehicle spawn probability and dynamically modify green and red light durations.

3. Discussions

The development of the Traffic Light System (TLS) presented several integrated challenges, ranging from low-level hardware configuration to high-level RTOS task management. The following sections detail the primary obstacles encountered and the solutions implemented.

3.1 Hardware Integration

Initial challenges focused on the GPIO and ADC initialization. Navigating the STM32 Standard Peripheral Library took a long time, and we spent a lot of time watching the Brightspace videos and reading documentation.

A significant portion of the debugging phase was dedicated to "hardware vs. software" isolation. Upon successful ADC configuration, the system initially reported values near 60,000, which was unexpected for a 12-bit converter (expected range: 0–4095). Troubleshooting revealed two distinct hardware issues:

- Grounding: The potentiometer was connected to the ground rail, but the rail itself was not bridged to the microcontroller's ground, leading to floating, erratic values.
- Voltage Logic: After fixing the ground, the values peaked at 2,000. We identified that the internal resistance in the circuit was preventing the ADC from reaching the full dynamic range. Removing the resistor allowed the potentiometer to utilize the full 0–4095 range, providing ideal and dependable values for traffic flow control.

3.2 Task Starvation and Priority Tuning

One of the most critical software bugs encountered was Task Starvation. To ensure the system was responsive to user input, the Traffic Flow Adjustment Task was initially assigned the highest priority with a very short delay (100ms).

However, because this task was "greedy" and ran so frequently, the FreeRTOS scheduler prioritized it exclusively, preventing lower-priority tasks (like car movement and display updates) from running, and the system looked halted. This resulted in a system where the potentiometer worked, but the traffic simulation remained frozen.

Resolution: We realized that human interaction with a potentiometer does not require sub-millisecond latency. By increasing task priority and increasing the delay to 1 second,

we eliminated the starvation. This provided a smooth user experience where changes in flow were still perceived as "instant" by the operator without blocking the rest of the application.

3.3 Software Architecture and Refactoring

We started implementing the system with only two tasks. However, the System Display Task became the primary task, handling both the complex logic of car movement and the physical shifting of the LEDs.

To improve modularity, we refactored the design to include a third task dedicated to Traffic Density Calculations. We implemented an additional FreeRTOS Queue to pass this data. This separation of concerns allowed for:

- More robust randomization in the Traffic Density Task.
- A simpler and more understandable update to the car array in the Display Task.
- Clearer debugging, as logic issues were isolated from display issues.

3.4 Logic Control and State Management

We encountered a logical race condition in the Traffic Light timer callback. Initially, the timer callback was overwriting the current state variable before the transition logic could execute, causing the lights to skip phases or stall in a single color.

We resolved this by implementing variables for the current state and the next state. This ensured that transitions (Green → Yellow → Red) occurred only after the current state's timing requirements were fully satisfied, meeting the project's functional requirements for light durations.

4. Limitations and Possible Improvements

Although the system met all functional and technical requirements outlined in the lab manual, several practical limitations were observed in both the implementation and overall design.

The system models only a single-lane, single-direction road with one intersection and no turning movements. As a result, the traffic logic is quite simple. Vehicles either move forward, or stop at a single stop line. There are no opposing flows, cars waiting to turn, or pedestrian interaction. This simplifies state management and scheduling but does not reflect the complexity of real-world intersections.

The vehicle behaviour is also uniform. All cars move at the same speed, maintain fixed spacing constraints, and do not account for acceleration, reaction time, or braking effects beyond blocking at the single stop line.

Possible improvements include extending the model to support multiple lanes or bidirectional traffic, implementing independent light control for different directions, or introducing more advanced vehicle behaviour such as variable speeds. These extensions would require additional tasks or more complex state management but would result in a more realistic traffic simulation.

5. Summary

This project successfully implemented a real-time Traffic Light System using the STM32F4 Discovery board and FreeRTOS. The final design satisfies the functional and technical requirements specified in the lab manual, including randomized traffic generation proportional to the potentiometer input, timer-controlled traffic light states, and queue-based inter-task communication.

The system architecture consists of three FreeRTOS tasks, three message queues, and one software timer. The potentiometer dynamically adjusts both vehicle generation rate and traffic light timing, while vehicles move at a constant speed and obey the traffic signal constraints.

Through the implementation process, challenges related to ADC configuration, task scheduling, and state transitions were identified and resolved. The final system demonstrates stable real-time behaviour and effective use of FreeRTOS features in an embedded control application.

Overall, the project provided practical experience in designing, debugging, and refining a multi-task real-time embedded system using structured inter-task communication and timer-based control logic.

6. Appendix

The appendix displays our source code for the traffic light project.

```
/* Standard includes. */
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include "stm32f4_discovery.h"
#include "../Libraries/STM32F4xx_StdPeriph_Driver/inc/stm32f4xx_gpio.h"
#include "../Libraries/STM32F4xx_StdPeriph_Driver/inc/stm32f4xx_adc.h"
```

```c
#include "../Libraries/STM32F4xx_StdPeriph_Driver/inc/stm32f4xx_rcc.h"

/* Kernel includes. */
#include "stm32f4xx.h"
#include "../FreeRTOS_Source/include/FreeRTOS.h"
#include "../FreeRTOS_Source/include/queue.h"
#include "../FreeRTOS_Source/include/semphr.h"
#include "../FreeRTOS_Source/include/task.h"
#include "../FreeRTOS_Source/include/timers.h"

/*-----------------------------------------------------------*/

#define QUEUE_LENGTH 1

#define RED_LIGHT      GPIO_Pin_0
#define YELLOW_LIGHT   GPIO_Pin_1
#define GREEN_LIGHT    GPIO_Pin_2

#define CLOCK          GPIO_Pin_7
#define RESET          GPIO_Pin_8
#define DATA           GPIO_Pin_6

#define POT            GPIO_Pin_3

typedef enum { STATE_GREEN, STATE_YELLOW, STATE_RED } LightState;

// Queues and Timers
QueueHandle_t xFlowQueue;
QueueHandle_t xLightQueue;
QueueHandle_t xSpawnQueue;
TimerHandle_t xLightTimer;

/* Function Prototypes */
static void prvSetupHardware(void);
static void vInitializeGPIO(void);
static void vInitializeADC(void);
static void vLightTimerCallback(TimerHandle_t xTimer);
uint32_t calculate_traffic_light_duration(uint16_t adc_val, LightState state);
static void move_cars(int *car);
static void shift_in_intersection(int *car_array, LightState current_light);
static void shift_after_intersection(int *car_array);
static void shift_before_intersection(int *car_array, LightState current_light);

/* Tasks */
void vTrafficFlowTask(void *pvParameters);
```

```c
void vDisplayTask(void *pvParameters);
void vTrafficDensityTask(void *pvParameters);

/*-----------------------------------------------------------*/

int main(void)
{
    prvSetupHardware();

    // Create Queues
    xFlowQueue = xQueueCreate(QUEUE_LENGTH, sizeof(uint16_t)); // POT queue
    xLightQueue = xQueueCreate(QUEUE_LENGTH, sizeof(LightState));
    xSpawnQueue = xQueueCreate(QUEUE_LENGTH, sizeof(uint16_t));

    // Add to registry for debugging
    vQueueAddToRegistry(xFlowQueue, "FlowQueue");
    vQueueAddToRegistry(xLightQueue, "LightQueue");
    vQueueAddToRegistry(xSpawnQueue, "SpawnQueue");

    // Create timer
    xLightTimer = xTimerCreate("TrafficLightTimer", pdMS_TO_TICKS(2000), pdFALSE,
(void*)0, vLightTimerCallback);

    // Create tasks
    xTaskCreate(vTrafficFlowTask, "Flow", 256, NULL, 2, NULL);
    xTaskCreate(vDisplayTask, "Display", 256, NULL, 1, NULL);
    xTaskCreate(vTrafficDensityTask, "Density", 256, NULL, 1, NULL);

    // Start timers and tasks
    xTimerStart(xLightTimer, 0);
    vTaskStartScheduler();

    return 0;
}

/*-----------------------------------------------------------*/

// Reads Potentiometer and updates the Flow Queue
void vTrafficFlowTask(void *pvParameters) {
    uint16_t adc_val;
    for(;;) {
        ADC_SoftwareStartConv(ADC1);
        while(!ADC_GetFlagStatus(ADC1, ADC_FLAG_EOC));
        adc_val = ADC_GetConversionValue(ADC1);
```

```
        // Update queue with ADC
        xQueueOverwrite(xFlowQueue, &adc_val);

        vTaskDelay(pdMS_TO_TICKS(1000)); // Sample every 1000ms
    }
}

void vTrafficDensityTask(void *pvParameters) {
    uint16_t adc_val;
    uint16_t next_car;
    for(;;) {
        if (xQueuePeek(xFlowQueue, &adc_val, 0) == pdTRUE) {

            uint32_t rand_val = rand() % 100; // Get a value between 0-99

            // 100/18 = 5.55, so at a minimum, 1 car every 5 seconds
            uint32_t threshold = 18 + (adc_val * (100 - 18) / 4095);

            // When highest density threshold is 100, rand_val is always less
(always get a 1).
            // When minimum threshold is 18, there is only an 18% chance of getting
a car
            // spawned (approx. 1 car every 5 seconds).
            if (rand_val < threshold) {
                next_car = 1;
            } else {
                next_car = 0;
            }

            xQueueOverwrite(xSpawnQueue, &next_car);
        }
        vTaskDelay(pdMS_TO_TICKS(1000));
    }
}

/*-----------------------------------------------------------*/

void vDisplayTask(void *pvParameters) {
    LightState current_light = STATE_GREEN;
    uint16_t next_car_signal = 0;
    int car_array[24] = {0};

    for(;;) {
        xQueuePeek(xLightQueue, &current_light, 0);  // Puts the value in the queue
into var
```

```
        xQueueReceive(xSpawnQueue, &next_car_signal, 0);

        shift_after_intersection(car_array);
        shift_in_intersection(car_array, current_light);
        shift_before_intersection(car_array, current_light);

        car_array[0] = next_car_signal; // Spawn new car based on randomness and
traffic density

        move_cars(car_array);

        vTaskDelay(pdMS_TO_TICKS(1000)); // Move cars every second
    }
}

/*-----------------------------------------------------------*/

// Update light states
void vLightTimerCallback(TimerHandle_t xTimer) {
    static LightState next_state = STATE_GREEN;
    static LightState cur_state = STATE_GREEN;

    uint16_t adc_val;
    xQueuePeek(xFlowQueue, &adc_val, 0);

    GPIO_ResetBits(GPIOC, RED_LIGHT | YELLOW_LIGHT | GREEN_LIGHT);

    switch(next_state) {
        case STATE_GREEN:
            cur_state = STATE_GREEN;
            GPIO_SetBits(GPIOC, GREEN_LIGHT);
            next_state = STATE_YELLOW;
            xTimerChangePeriod(xTimer,
pdMS_TO_TICKS(calculate_traffic_light_duration(adc_val, STATE_GREEN)), 0);
            break;
        case STATE_YELLOW:
            cur_state = STATE_YELLOW;
            GPIO_SetBits(GPIOC, YELLOW_LIGHT);
            next_state = STATE_RED;
            xTimerChangePeriod(xTimer, pdMS_TO_TICKS(2000), 0); // Yellow is
constant at 2 secs
            break;
        case STATE_RED:
            cur_state = STATE_RED;
            GPIO_SetBits(GPIOC, RED_LIGHT);
```

```
            next_state = STATE_GREEN;
            xTimerChangePeriod(xTimer,
pdMS_TO_TICKS(calculate_traffic_light_duration(adc_val, STATE_RED)), 0);
            break;
    }
    xQueueOverwrite(xLightQueue, &cur_state);
}

/*-----------------------------------------------------------*/

static void shift_after_intersection(int *car_array) {
    for (int i = 20; i >= 12; i--) {
        car_array[i] = car_array[i - 1];
    }
    car_array[12] = 0;
}

/*-----------------------------------------------------------*/

static void shift_in_intersection(int *car_array, LightState current_light) {
    // Move the car at the exit of the intersection (11) into the post-intersection
(12)
    car_array[12] = car_array[11];

    // Shift cars within the intersection
    for (int i = 11; i >= 9; i--) {
        car_array[i] = car_array[i - 1];
    }

    car_array[9] = 0;
}

/*-----------------------------------------------------------*/

static void shift_before_intersection(int *car_array, LightState current_light) {
    // Check if the car at the front (index 8) can enter the intersection (index 9)
    if (current_light == STATE_GREEN) {
        car_array[9] = car_array[8];
        car_array[8] = 0; // It moved forward
    }
    // If Red or Yellow, car_array[8] stays put (blocking others)

    // Shift the rest of the pre-intersection queue
    for (int i = 8; i >= 0; i--) {
        // Skip index 7
```

```
        if (i == 7) continue;

        // Only move forward if the spot ahead is empty
        if (car_array[i] == 0) {
            if (i - 1 == 7) {
                car_array[i] = car_array[6];
                car_array[6] = 0;
            } else {
                car_array[i] = car_array[i - 1];
                car_array[i - 1] = 0;
            }
        }
    }
}

/*-------------------------------------------------------------*/

uint32_t calculate_traffic_light_duration(uint16_t adc_val, LightState state) {
    // Range: 2s to 4s based on 12-bit ADC (0-4095)
    if (state == STATE_GREEN) {
        return 2000 + (adc_val * 2000 / 4095); // Large ADC results in long green
    } else {
        return 4000 - (adc_val * 2000 / 4095); // Large ADC results in short red (2
seconds)
    }
}
/*-------------------------------------------------------------*/

static void move_cars(int *car_array) {
    GPIO_ResetBits(GPIOC, RESET);
    GPIO_SetBits(GPIOC, RESET);

    for(int i = 23; i >= 0; i--) {
        if(car_array[i] == 1) {
            GPIO_SetBits(GPIOC, DATA);
        } else {
            GPIO_ResetBits(GPIOC, DATA);
        };
        GPIO_ResetBits(GPIOC, CLOCK);
        GPIO_SetBits(GPIOC, CLOCK);
        GPIO_ResetBits(GPIOC, CLOCK);
    }
}

/*-------------------------------------------------------------*/
```

```
void vApplicationMallocFailedHook(void)
{
    /* The malloc failed hook is enabled by setting
    configUSE_MALLOC_FAILED_HOOK to 1 in FreeRTOSConfig.h.

    Called if a call to pvPortMalloc() fails because there is insufficient
    free memory available in the FreeRTOS heap. pvPortMalloc() is called
    internally by FreeRTOS API functions that create tasks, queues, software
    timers, and semaphores. The size of the FreeRTOS heap is set by the
    configTOTAL_HEAP_SIZE configuration constant in FreeRTOSConfig.h. */
    for(;;);
}

/*-----------------------------------------------------------*/

void vApplicationStackOverflowHook(xTaskHandle pxTask, signed char *pcTaskName)
{
    (void) pcTaskName;
    (void) pxTask;

    /* Run time stack overflow checking is performed if
    configCHECK_FOR_STACK_OVERFLOW is defined to 1 or 2. This hook
    function is called if a stack overflow is detected. pxCurrentTCB can be
    inspected in the debugger if the task name passed into this function is
    corrupt. */
    for(;;);
}

/*-----------------------------------------------------------*/

void vApplicationIdleHook(void)
{
    volatile size_t xFreeStackSpace;

    /* The idle task hook is enabled by setting configUSE_IDLE_HOOK to 1 in
    FreeRTOSConfig.h.

    This function is called on each cycle of the idle task. In this case it
    does nothing useful, other than report the amount of FreeRTOS heap that
    remains unallocated. */
    xFreeStackSpace = xPortGetFreeHeapSize();

    if(xFreeStackSpace > 100)
    {
```

```
        /* By now, the kernel has allocated everything it is going to, so
        if there is a lot of heap remaining unallocated then
        the value of configTOTAL_HEAP_SIZE in FreeRTOSConfig.h can be
        reduced accordingly. */
    }
}

/*-----------------------------------------------------------*/

static void prvSetupHardware(void)
{
    /* Ensure all priority bits are assigned as preemption priority bits.
    http://www.freertos.org/RTOS-Cortex-M3-M4.html */
    NVIC_SetPriorityGrouping(0);

    // Set up clocks pg. 390 ref manual
    RCC_AHB1PeriphClockCmd(RCC_AHB1Periph_GPIOC, ENABLE);
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_ADC1, ENABLE);

    // Set up GPIO and ADC
    vInitializeGPIO();
    vInitializeADC();
}

/*-----------------------------------------------------------*/

static void vInitializeGPIO(void)
{
    GPIO_InitTypeDef GPIO_InitStruct;

    /* Traffic Lights + Shift Register */
    GPIO_InitStruct.GPIO_Pin = RED_LIGHT | YELLOW_LIGHT | GREEN_LIGHT | CLOCK |
RESET | DATA;
    GPIO_InitStruct.GPIO_Mode  = GPIO_Mode_OUT;
    GPIO_InitStruct.GPIO_OType = GPIO_OType_PP;
    GPIO_InitStruct.GPIO_PuPd  = GPIO_PuPd_NOPULL;
    GPIO_InitStruct.GPIO_Speed = GPIO_Speed_100MHz;

    GPIO_Init(GPIOC, &GPIO_InitStruct);

    /* POT */
    GPIO_InitStruct.GPIO_Pin  = POT;
    GPIO_InitStruct.GPIO_Mode = GPIO_Mode_AN;
    GPIO_InitStruct.GPIO_PuPd = GPIO_PuPd_NOPULL;
```

```
    GPIO_Init(GPIOC, &GPIO_InitStruct);
}

/*-------------------------------------------------------------*/

static void vInitializeADC(void)
{
    ADC_InitTypeDef ADC_InitStruct;

    ADC_StructInit(&ADC_InitStruct);
    ADC_InitStruct.ADC_Resolution = ADC_Resolution_12b; // 12-bit resolution for
the potentiometer (2^12)
    ADC_InitStruct.ADC_ScanConvMode = DISABLE;           // Single channel so
disable
    ADC_InitStruct.ADC_ContinuousConvMode = DISABLE;     // Only when called so
disable
    ADC_InitStruct.ADC_ExternalTrigConvEdge = ADC_ExternalTrigConvEdge_None;
    ADC_InitStruct.ADC_DataAlign = ADC_DataAlign_Right;
    ADC_InitStruct.ADC_NbrOfConversion = 1;

    ADC_Init(ADC1, &ADC_InitStruct);
    ADC_Cmd(ADC1, ENABLE);
    ADC_RegularChannelConfig(ADC1, ADC_Channel_13, 1, ADC_SampleTime_144Cycles);
}
```

References

[1] Department of Electrical and Computer Engineering, ECE 455: Real Time Computer Systems Design Project – Lab Manual. University of Victoria, Victoria, BC, Canada, Nov. 2019.